AI TOOL PROMPT ENGINEERING REPORT

Submitted by:

Josh Clarence R. Agulo

BS INFORMATION TECHNOLOGY 2-A

GE 0000 16-607

March 2026

AI Tool Overview

Name of AI Tool

Perplexity AI

Purpose / Domain

Information Retrieval and Search

Why I Chose This Tool

I chose Perplexity AI for this exercise because it effectively combines large language models with real-time web search. As an IT student, I often need to find high-quality educational resources from across the internet. Unlike traditional search engines that provide a simple list of links, Perplexity synthesizes information from multiple sources into a coherent and cited response. This makes it a valuable tool for academic research and for discovering structured learning paths.

The platform is ideal for testing different prompting strategies. It understands context and provides nuanced recommendations, allowing me to see how specific inputs change the quality of search results. Furthermore, the citation system allows me to verify every source. This is essential for maintaining academic integrity. By experimenting with different prompt formulations, I can observe how specificity and structure influence the depth and organization of the information provided.

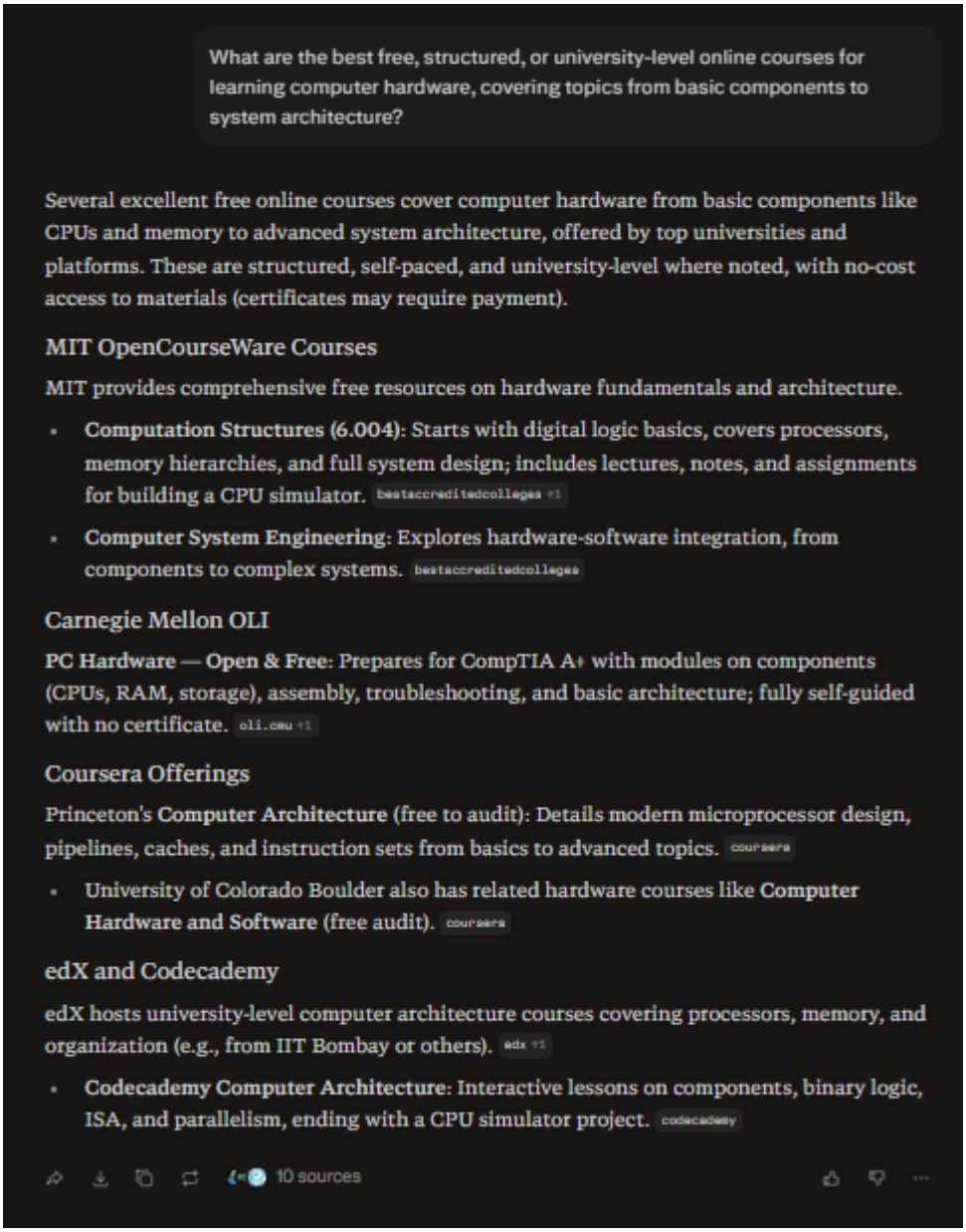
Ultimately, Perplexity AI served as a perfect testing ground. It demonstrated that a well-crafted prompt does more than just return relevant results; it ensures the information is presented in a way that is immediately useful for complex projects. Moving forward, I will use these structured prompting techniques to better navigate the tool's ability to synthesize high-level data.

Original Prompt

Initial Prompt

What are the best free, structured, or university-level online courses for learning computer hardware, covering topics from basic components to system architecture?

Output Screenshot



AI-Assisted Prompt

LLM Used for Improvement

ChatGPT Free (GPT-4o mini)

Improved Prompt

Act as a computer engineering curriculum designer. Create a structured, university-level learning path for mastering computer hardware—from beginner to advanced (basic components → digital logic → computer organization → system architecture → low-level design).

Recommend only free, high-quality online courses (e.g., Coursera, edX, MIT OpenCourseWare, university YouTube lectures, or similar). Prioritize courses that are equivalent to real university classes and include lectures, assignments, or hands-on labs.

For each recommended course, provide:

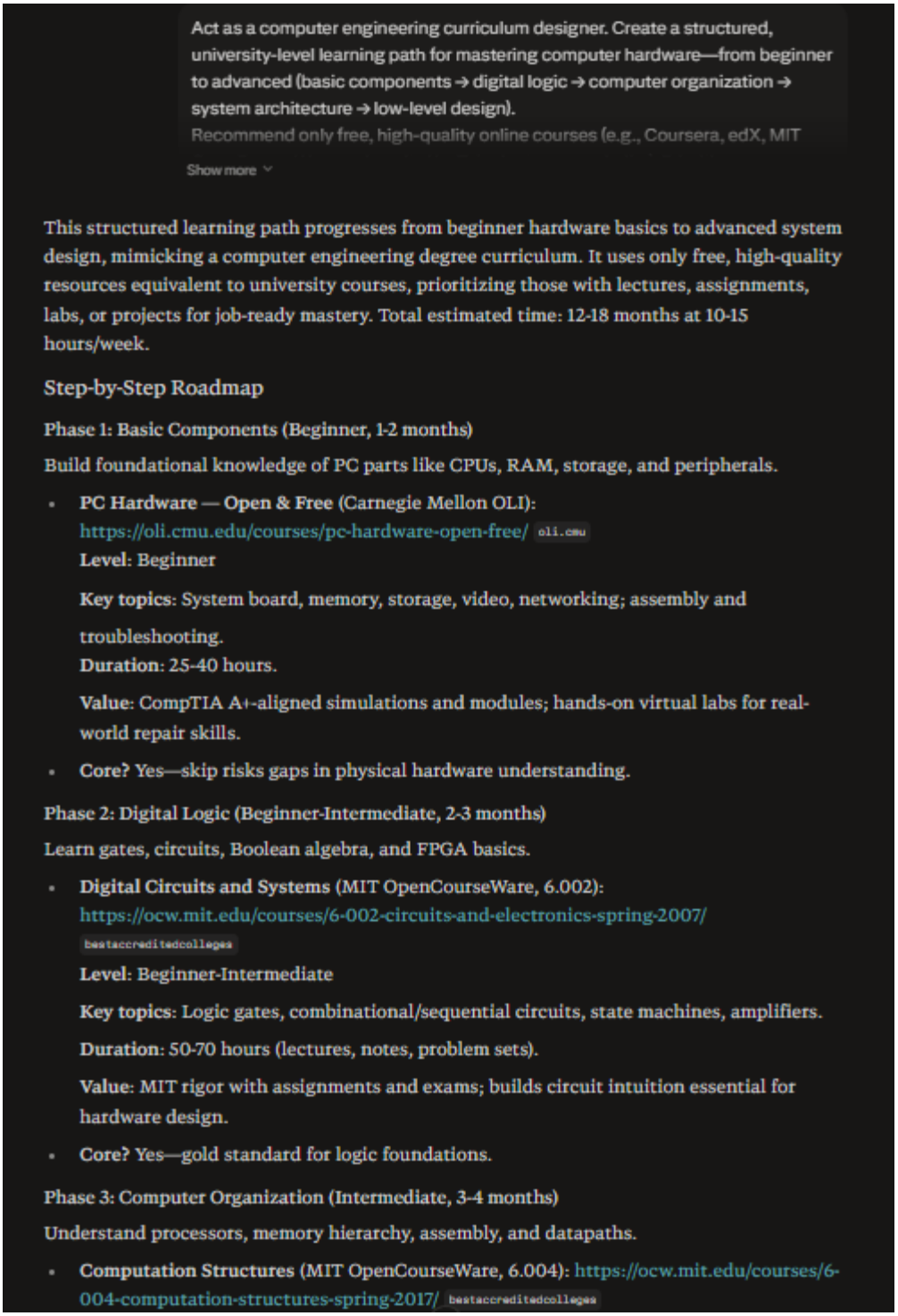
- *Course title and provider (with link)*
- *Level (beginner/intermediate/advanced)*
- *Key topics covered*
- *Estimated duration*
- *Why it's valuable (e.g., depth, projects, reputation)*

Then:

- *Organize the courses into a clear step-by-step roadmap (like a degree progression).*
- *Highlight the best "core" courses that should not be skipped.*
- *Suggest optional hands-on tools or simulators (e.g., building CPUs, circuits, etc.).*
- *Include any well-known "gold standard" courses (e.g., courses where you build a computer from scratch or simulate hardware systems).*

Assume I am a beginner aiming to reach an advanced, job-ready understanding of computer hardware and architecture.

Output Screenshot



Level: Intermediate

Key topics: Digital abstraction, processors, memory, assembly language, CPU simulator project.

Duration: 80-100 hours (lectures, labs, assignments).

Value: "Build a computer from scratch" via labs (NAND-to-Tetris style); legendary for hands-on mastery.

- **Core?** Yes—gold standard; do not skip.

Phase 4: System Architecture (Intermediate-Advanced, 3-4 months)

Dive into pipelining, caches, parallelism, and performance.

- **Computer Architecture** (Princeton University via Coursera, free audit): https://www.coursera.org/learn/comparch coursera

  Level: Intermediate-Advanced

  Key topics: ISA, pipelining, caches, virtual memory, multiprocessors.

  Duration: 40-60 hours (videos, quizzes, programming labs).

  Value: David Wentzlaff's labs simulate real architectures; Princeton depth prepares for industry roles. reddit

- **Core?** Yes.

Phase 5: Low-Level Design & Advanced Topics (Advanced, 3-4 months)

Integrate hardware-software; explore modern systems.

- **Computer System Architecture** (Saylor Academy, CS301): https://learn.saylor.org/course/view.php?id=71 learn.saylor

  Level: Advanced

  Key topics: CPU design, I/O, multiprocessing, RISC-V labs.

  Duration: 48 hours (readings, quizzes, projects).

  Value: Self-paced with certificate option; bridges to embedded systems.

- **Computer System Engineering** (MIT OpenCourseWare, 6.033): https://ocw.mit.edu/courses/6-033-computer-system-engineering-spring-2018/ bestaccreditedcolleges

  Level: Advanced

  Key topics: Hardware-software interface, fault tolerance, security.

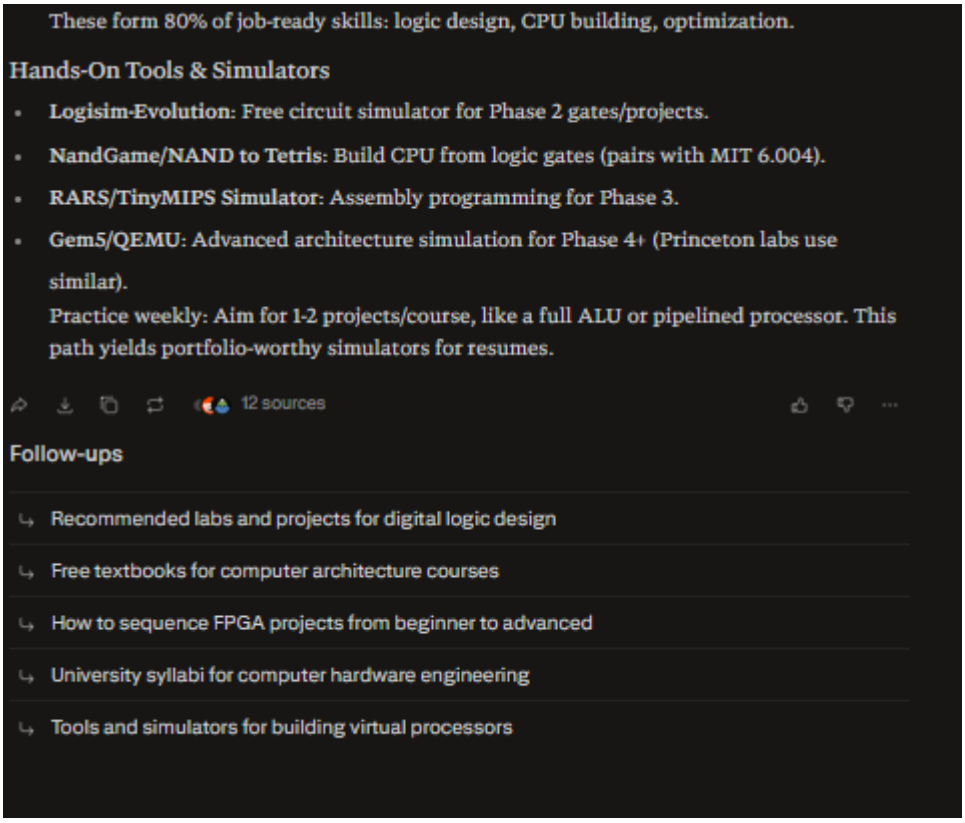
  Duration: 60-80 hours.

  Value: Case studies and projects on real systems; MIT's holistic view.

- **Core?** No—pick one based on interests (e.g., Saylor for labs).

Core Courses (Must-Do, ~6-9 months)

1. PC Hardware (OLI) oli.cmu
2. Digital Circuits (MIT 6.002) bestaccreditedcolleges
3. Computation Structures (MIT 6.004)—gold standard "from NAND to Tetris" reddit +1
4. Computer Architecture (Princeton Cours) coursera



Comparison and Analysis

Comparing the outputs from my original and AI-improved prompts revealed a substantial difference in quality. While my first attempt was clear, it produced a generic list of online courses. The AI's response was a simple enumeration of popular platforms that lacked the structural depth and organization I needed. It was useful but scattered, leaving me to piece together a coherent learning path on my own.

In contrast, the improved prompt generated comprehensive and actionable results. By assigning the AI a specific role, such as a computer engineering curriculum designer, I shifted the output from a basic list to a structured academic plan. This new version provided a clear progression from foundational concepts to advanced topics. By requesting specific metadata like course duration and value propositions, I received a much richer set of information that explained not just what to learn but why it mattered.

The improved prompt also enhanced clarity by defining a strict format. Instead of leaving the structure to chance, I prescribed exactly what details to include for each course. This eliminated ambiguity. The hierarchical approach forced the AI to think systematically

about how topics connect. Furthermore, adding constraints like university-level content and hands-on labs filtered out superficial resources. This ensured the recommendations remained relevant to my goal of gaining a job-ready understanding.

Specific strategic changes drove these improvements. First, establishing a professional persona framed the task as an expert design challenge. Second, requesting a step-by-step roadmap transformed the data into a navigable path. Third, using a structured template ensured the AI filled in every necessary detail. Finally, providing context about my beginner status allowed the AI to calibrate the difficulty level appropriately.

This exercise proved that effective prompting is about reducing uncertainty. High-quality prompts act as a contract that specifies the desired output, constraints, and quality criteria. The most powerful technique I discovered was role assignment. This shifts the AI from simple question-answering to complex task execution. Moving forward, I will invest more time in crafting structured prompts to ensure the results align with my specific expectations.